

WHAT IS AIRFLOW?

Apache Airflow is an open-source workflow orchestration tool used to **schedule, monitor, and manage data pipelines**.

It allows you to define workflows as code using Python and automate tasks such as:

- Extracting data from databases
- Running ETL/ELT jobs
- Executing Spark jobs
- Running Databricks notebooks
- Loading data into data warehouses
- Sending email notifications

WHY AIRFLOW INSTEAD OF USING AZURE DATA PIPELINES?

Airflow is preferred over Azure Data Pipelines when organizations need code-based workflow orchestration, complex dependency management, multi-cloud support, and greater flexibility. Airflow allows workflows to be defined in Python and integrated with DevOps practices, while Azure Data Pipelines is more suitable for Azure-centric, low-code data integration pipelines.

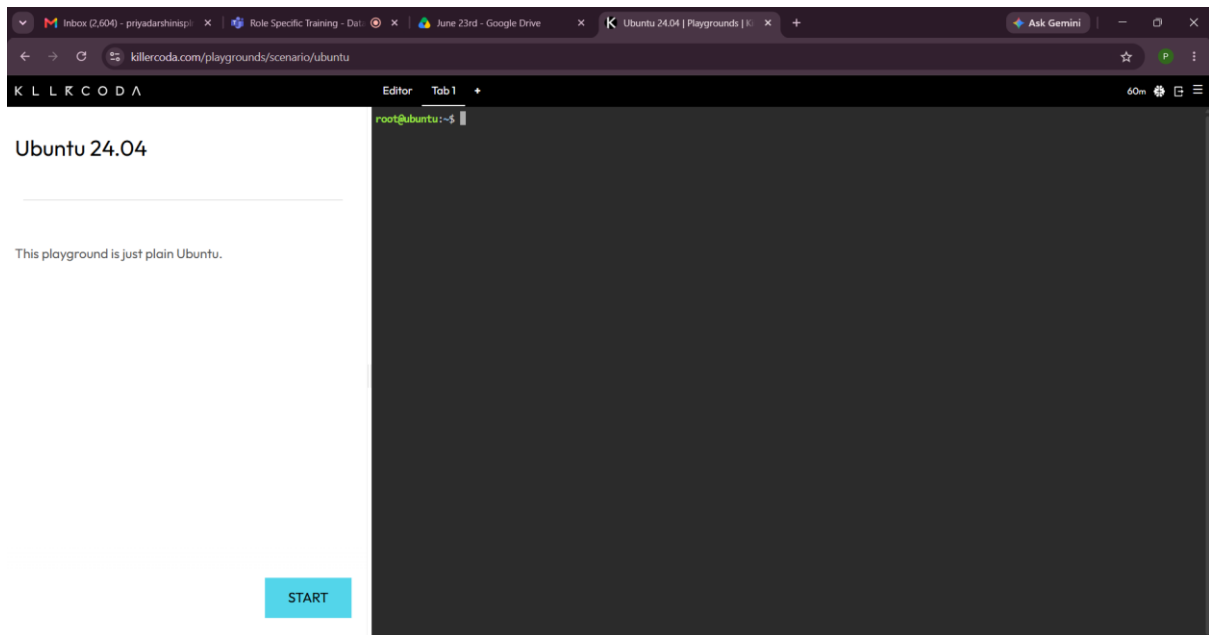
WHAT IS DAG?

DAG (Directed Acyclic Graph) is a collection of tasks and their dependencies that defines the execution flow of a workflow in Airflow. It ensures tasks run in the correct order without creating cycles.

EXECUTE ALL THE KILLER CODA STEPS.

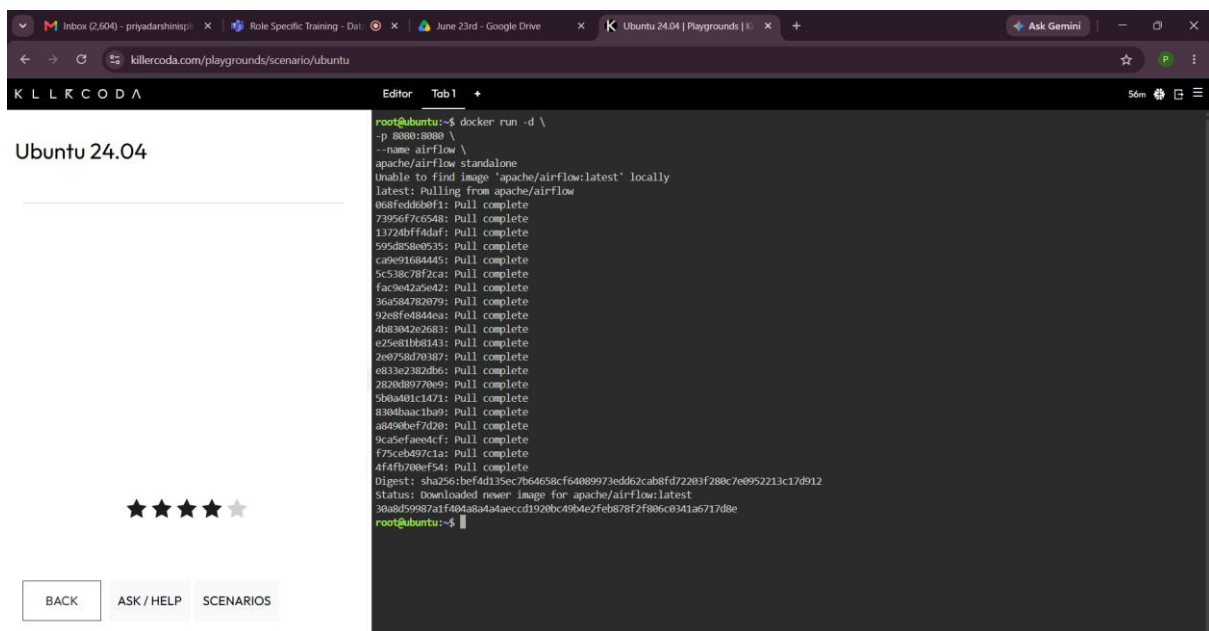
Open killercoda and click on playgrounds.

From playgrounds, select Ubuntu 24.04 and click start button



Then run,

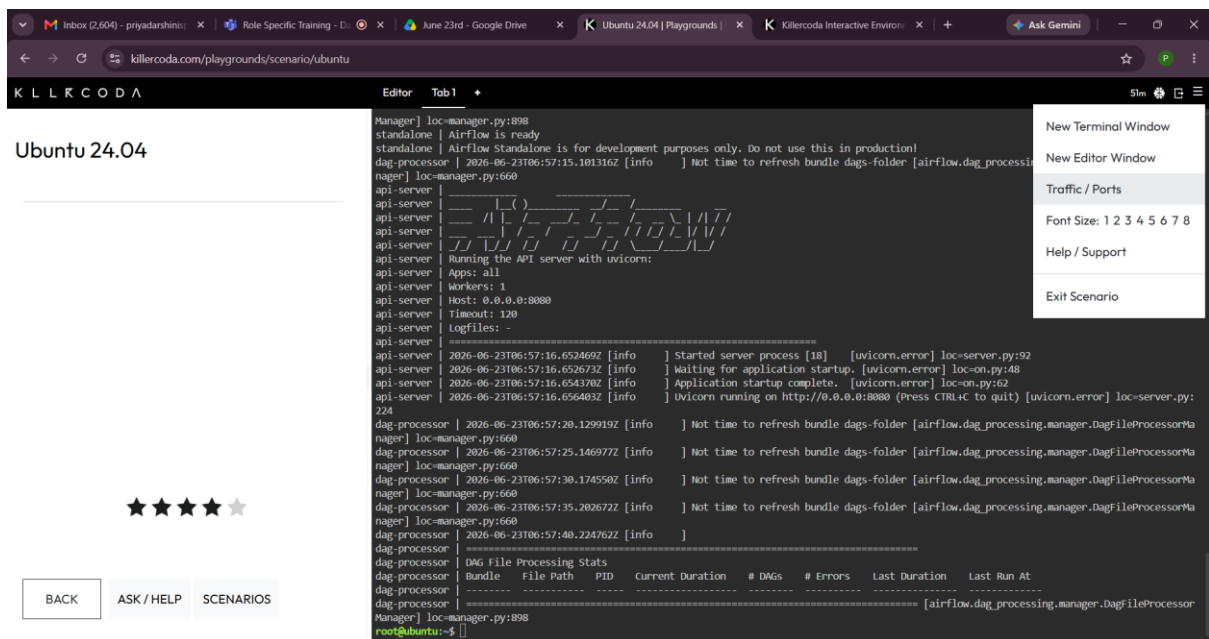
```
docker run -d \
-p 8080:8080 \
--name airflow \
apache/airflow standalone
```



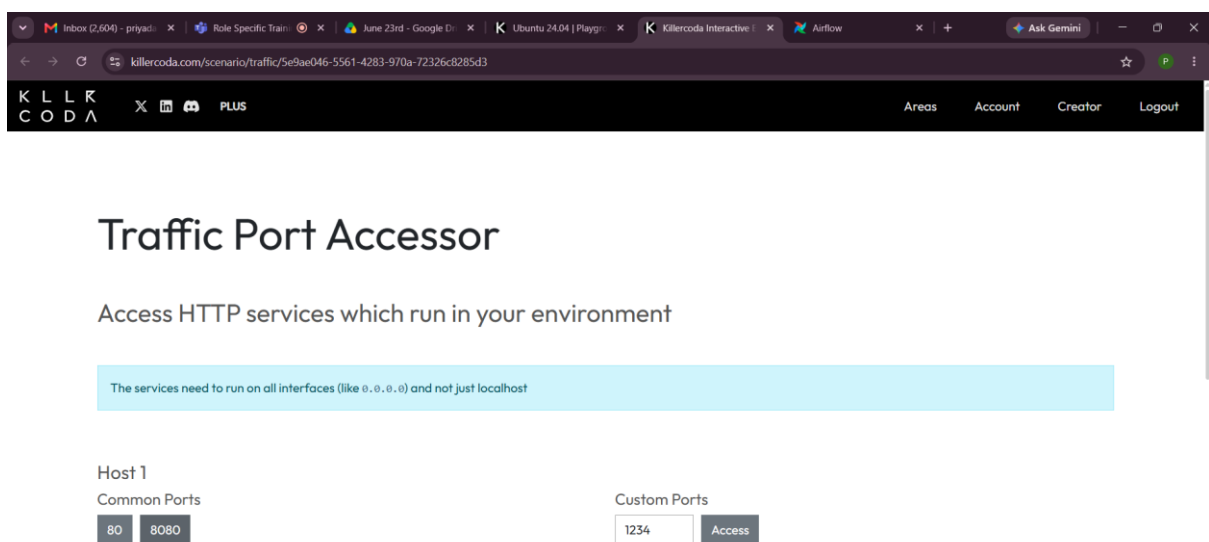
To check container,

```
docker ps
```

Click on the menu in the right top corner, then select **Traffic/Ports**.

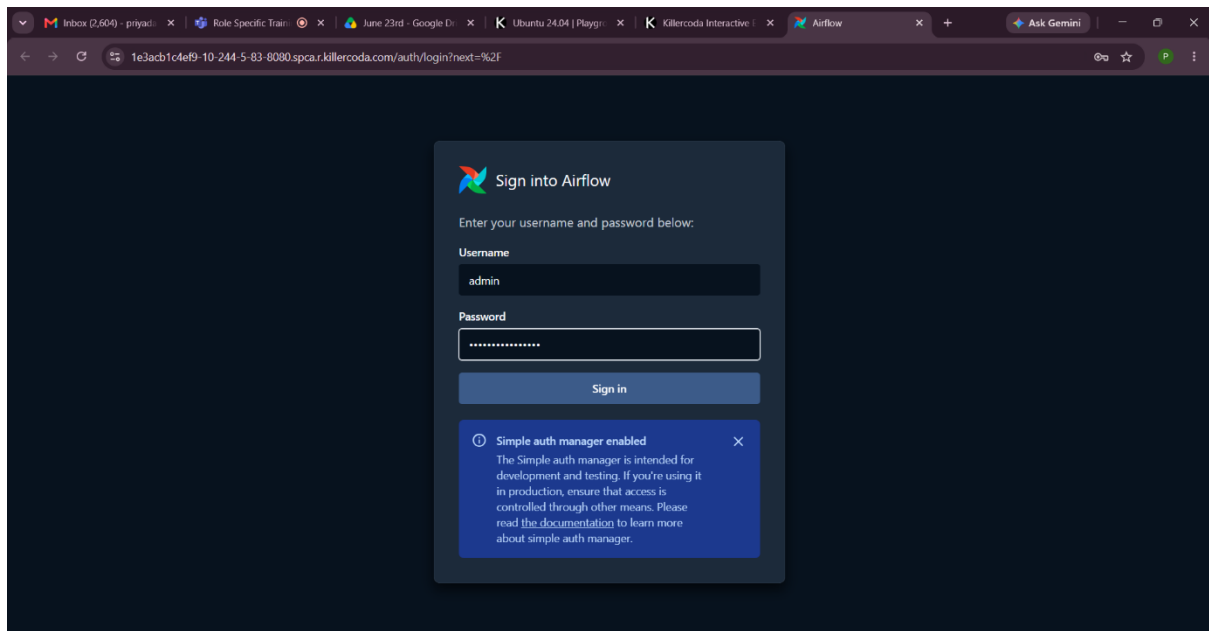


It will open Traffic Port Accessor, click on the common ports 8080 to open Airflow UI.



In Airflow UI,

Enter the username and password to login,



EXECUTE YOUR FIRST DAG.

In Ubuntu Terminal, run

```
docker exec -it airflow bash
```

```
cd /opt/airflow/dags
```

To open first_dag.py, run

```
cat > first_dag.py
```

```
from airflow.sdk import DAG
```

```
from airflow.providers.standard.operators.python import PythonOperator
```

```
from datetime import datetime
```

```
def task1_function():
```

```
    print("Task 1 Started")
```

```
def task2_function():
```

```
    print("Task 2 Running")
```

```
def task3_function():
```

```
    print("Task 3 Completed")
```

```
with DAG(
```

```
    dag_id="first_training_dag",
```

```
    start_date=datetime(2025, 1, 1),
```

```
schedule="@daily",
```

```
catchup=False,
```

```
tags=["training"]
```

) as dag:

```
task1 = PythonOperator(
```

```
    task_id="task1",
```

```
    python_callable=task1_function
```

```
)
```

```
task2 = PythonOperator(
```

```
    task_id="task2",
```

```
    python_callable=task2_function
```

```
)
```

```
task3 = PythonOperator(
```

```
    task_id="task3",
```

```
    python_callable=task3_function
```

```
)
```

```
task1 >> task2 >> task3
```

Then give ctrl + d it is to save

ls (to show the file is there or not), **airflow dags list-import-errors** (to check if there is any errors)

airflow dags reserialize

airflow dags list

To execute the dag, go to Airflow UI and run it by clicking trigger.

The screenshot displays the Airflow web interface. On the left sidebar, the 'Dags' menu is selected, showing a list of DAGs: 'first_training_dag', 'task1', 'task2', and 'task3'. The main panel shows the details for the 'first_training_dag' run, which is marked as 'Success'. The 'Task Instances' tab is active, displaying a table of task execution results.

Task ID	Map Index	State	Start Date	Try Number	Operator	Duration
task3		Success	2026-06-23 10:24:55	1	PythonOperator	00:00:01.268
task2		Success	2026-06-23 10:24:54	1	PythonOperator	00:00:00.275
task1		Success	2026-06-23 10:24:50	1	PythonOperator	00:00:06.363